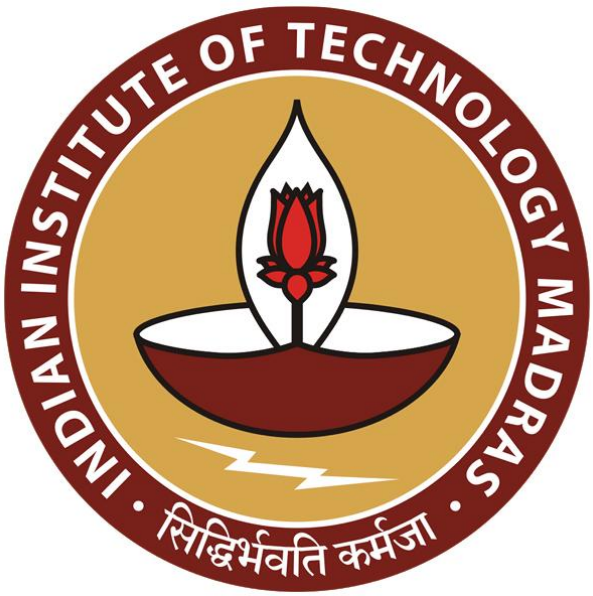


IIT Madras

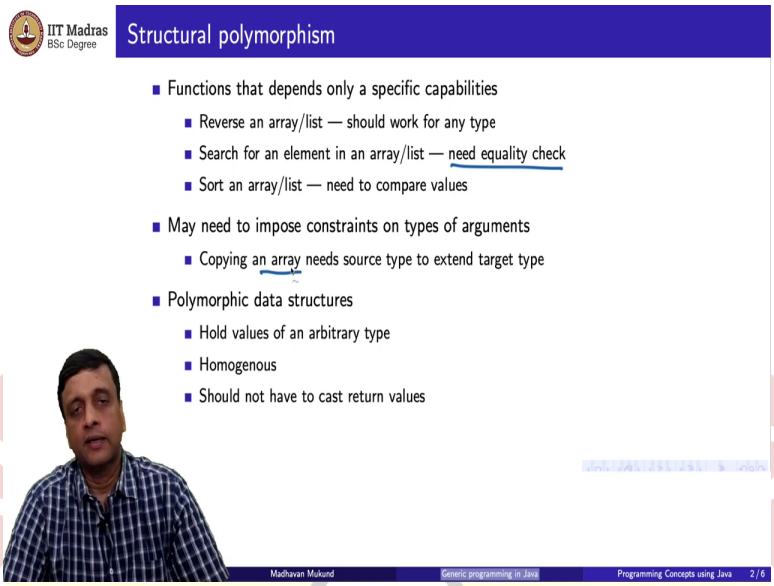
ONLINE DEGREE

Programming Concepts using Java
Professor. Madhavan Mukund
Department of Computer Science
Chennai Mathematical Institute
Indian Institute of Technology, Madras
Generic Programming in Java

(Refer Slide Time: 0:13)

So, we are discussing structural polymorphism. And we said that the way this is implemented in Java is through what Java calls generics or generic programming.

(Refer Slide Time: 0:21)



So, just remember what we meant by structural polymorphism. We want to define functions, for example, that depend only on specific capabilities. So, it could be something which requires no capabilities at all, like reversing a list where we do not care what the objects are, we just want to change the positions, we just want to move some cartons around, we do not have to look inside the cartons.

It could be searching for an element, in which case, I need to at least be able to check whether two objects are the same. So, I need at least an equality check. Or it could be something like sorting, in which case, I need to actually be able to compare values. But then, sometimes we have multiple arguments and we need to constrain the types of the multiple arguments. So, one example we saw was array copying.

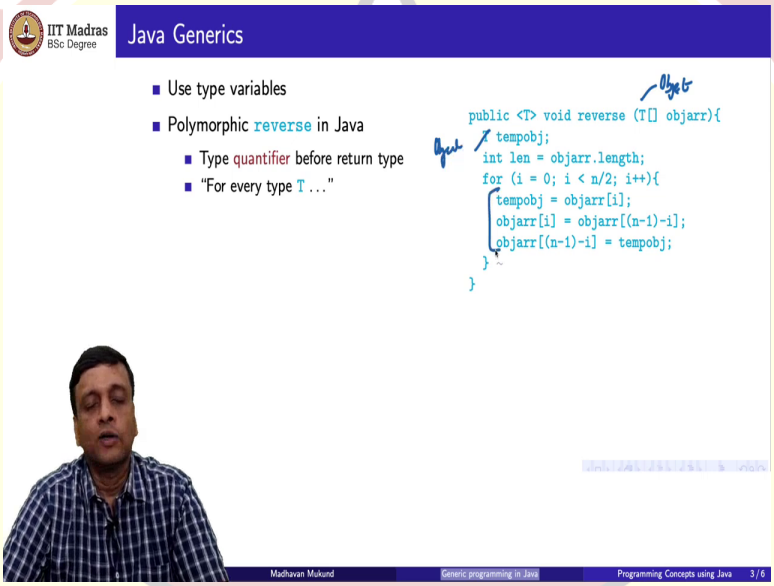
So, if I have a source array, and I want to copy it into a target array, in general, you would expect that the source array and the target array are the same type. But actually, more generally, in a language like Java, because we allow super type variables to have subtype values, we want that the source type is a subtype of the super type. So, if I say target i equal to source i, target i can be more general than source i. So, wherever I expect a ticket I am okay with an e-ticket, for example.

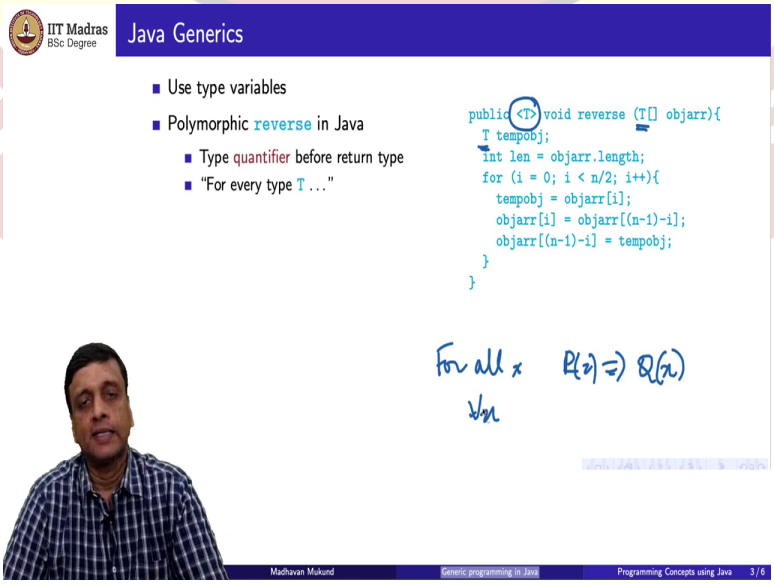
The other thing that we wanted to do was to have these polymorphic data structures. So, we want things like lists, arrays and so on to be able to take values of arbitrary type, but we also want it to

be homogeneous. So, we do not want to have mixed just because we allow arbitrary types, is arbitrary, but fixed. So, each array will have elements of one type, but we are not constraining what type that is.

And the other thing is that this type information should be preserved somehow in the array of the list. So, when we return a value or we continue, we do not want to keep casting things back and forth between a more general thing and a specific thing. So, these are all addressed, as we said, through this Java's notion of generics. So, let us take a look at what Java generics do.

(Refer Slide Time: 2:13)





Java Generics

- Use type variables
- Polymorphic `reverse` in Java
  - Type quantifier before return type
  - "For every type T ..."

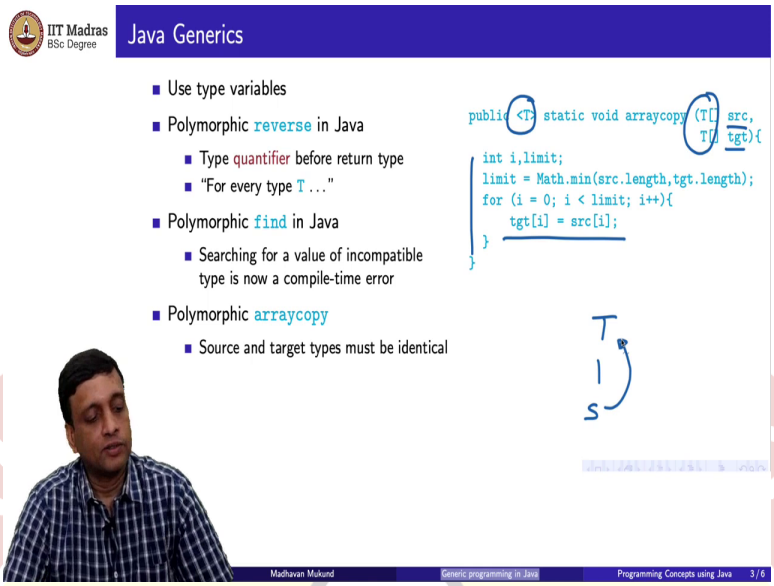
```
public <T> void reverse (T[] objarr){
  T tempobj;
  int len = objarr.length;
  for (i = 0; i < n/2; i++){
    tempobj = objarr[i];
    objarr[i] = objarr[(n-1)-i];
    objarr[(n-1)-i] = tempobj;
  }
}
```

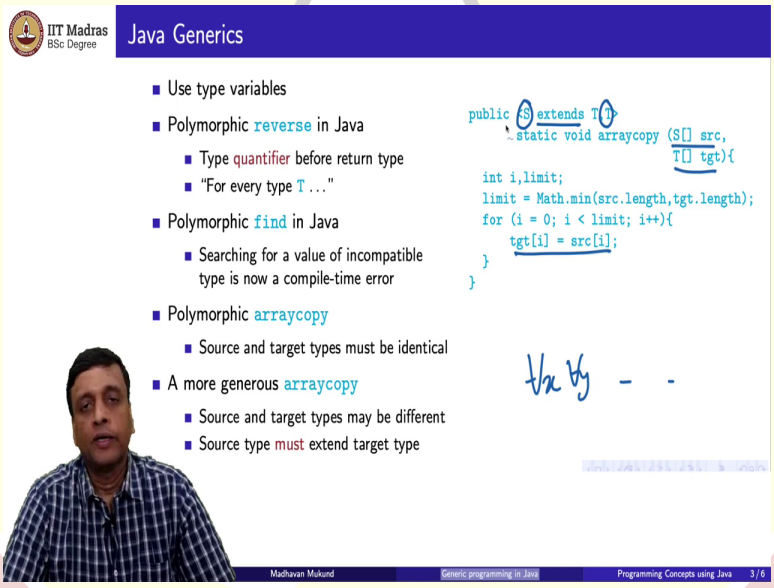
IIT Madras BSc Degree

Java Generics

- Use type variables
- Polymorphic `reverse` in Java
  - Type quantifier before return type
  - "For every type T ..."
- Polymorphic `find` in Java
  - Searching for a value of incompatible type is now a compile-time error

```
public <T> int find (T[] objarr, T o){
  int i;
  for (i = 0; i < objarr.length; i++){
    if (objarr[i] == o) {return i};
  }
  return (-1);
}
```





So, the main new feature that Java generics introduced into Java is this use of type variables. So, the easiest way to understand type variables is to look at the examples we have already seen and see how they are written in a generic form. So, let us start with this polymorphic reverse. So remember that earlier we had said that this was of type object and so was this temporary object which is used for swapping in this loop, so this is how we defined the earlier reverse.

So, now, what we are saying is that this array is of some arbitrary type T and the object that we need in order to achieve the swap is of the same type T. So, there is a notion of a type variable.

And this notation here, this angle bracket T, tells us that this function depends on this type variable.

So, in mathematical logic you sometimes say things like for all x, which is often written as for all x, something P of x implies Q of x. You write statement like this. For instance, you can say that for all x, if x is prime, and x is not 2, then x must be odd. So, any prime member other than 2 is an odd number. So, this is a kind of quantified statement. So, the x is now standing for some universe. In this case, x is talking about numbers, natural numbers. And you say for all natural numbers, if it is the case that x is a prime and x is not 2, then it is the case that x must be odd.

So, similarly, here you should think of this angle bracket as a type quantifier. It saying, for every type reverse is the function which takes an array of type T and reverses it and that type T is used inside. So, that is the main thing. So, the T that is used here, so whatever type of array you pass to reverse, automatically fixes the type of the object which is going to be used inside. So, the type T is, in this case, inferred.

So, it is defined for every type T, but you do not have to call it saying that I am calling reverse with an array of date or I am calling reverse with an array of ticket or so on. The moment you pass an array this type T is kind of matched to the type of the array that you pass and then internally it will run with an appropriate variable tempobj of that type.

Now, if I look at find, it is a very similar thing. Again, I have this type quantifier. So, find, remember, searches for an object inside an array. But now I have an array of type T and an object of type T and the rest is the same. I am just using quality. So, either this type T redefines this equality or it does not, in which case, I just use the capital object, capital O object equality, which is just the, that they refer to the same memory location. But I get now an extra bonus, which we did not have when we wrote it in this using the class hierarchy.

So, earlier, we had said that this was an object and this is also an object, but there was no guarantee that the two objects are of the same type. Now, fortunately, this equality would ensure that any way two objects of incomparable type would not get compared. So, the equality has this

built in feature that if I throw two objects of different type at it, it will always come out to be unequal.

But, as a bonus, now, what we can guarantee is that if somebody calls this find with two incompatible things, you will actually get a compiler error. So, it is more likely to be a mistake, if I asked you to search for a date in an array of tickets or something, than a feature. So, if you actually are searching for an object of one type in an array of objects of another type, most likely you made a mistake. And here fortunately, because I have now constrained that this type of array must match this type of object that I am looking for, actually a compile time, I can check that I am not doing an inconsistency. So, even though it does not affect the functioning of find at all, it is still a useful check to make sure that this function is being called in a sensible way.

So, what about the other function that we had, array copy. So, here is a strict version of the array copy, which says that if I want to copy from source to target, then both must have the same type. So, I have a dependence on this quantified type. So, for any type T, I can take an array of type T, and copy from that source array into another area of type T, which is the target array, so the rest of the code does not change at all. And this assignment is always guaranteed to work because both target i and source i have exactly the same thing.

But remember we said that this is not the only type of array copy which makes sense in Java, in particular, because in Java, if I have T and if I have S, then I can take values of type S and assign them to variables of type T. So, a subclass can be used to instantiate a super-class variable definition. So, in this case, what I want to say is that the source array and the target array may have different types, but the types must be related by this subclass.

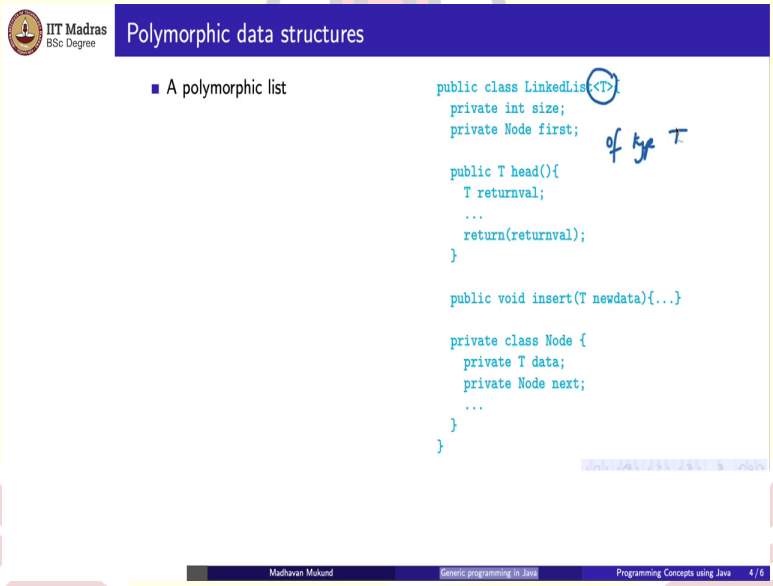
So, this is how Java writes it. It says that, so at the first point you have, inside this angle bracket, two variables. So, this is like saying some, in mathematics, like for all x, for all y to something. So, I have two different things they are independently quantified. But now I am specifying a dependence between them. I am saying they are not two arbitrary types, there is a constraint. So, the first type must extend the second type.

And what do I know? I know that the more specific type is in the source and the less specific type in the target. So, this assignment from target i, from source i to target i will always work,

because wherever I am expecting a variable of type T, I am okay with the variable of type S. So, this, now, through examples tells you how we use these type quantifiers and type variables to define functions.

And in particular, I can also use this extends predicate to define some kind of a constraint or dependency between two different types which might be used inside a function. And the other thing to notice that in the very first example, the type that we pass, so the type that we pass or we create as part of the function definition can be used inside the function to define some local variables that we need in order to execute the function that is had. So, we can derive this information at runtime in some sense. So, we can just say that this must be the same type as the type that the function was called with.

(Refer Slide Time: 9:17)



IIT Madras BSc Degree

Polymorphic data structures

- A polymorphic list
- The type parameter T applies to the class as a whole

```
public class LinkedList<T>{
  private int size;
  private Node first;

  public T head(){
    T returnval;
    ...
    return(returnval);
  }

  public void insert(T newdata){...}

  private class Node {
    private T data;
    private Node next;
    ...
  }
}
```


IIT Madras BSc Degree

Polymorphic data structures

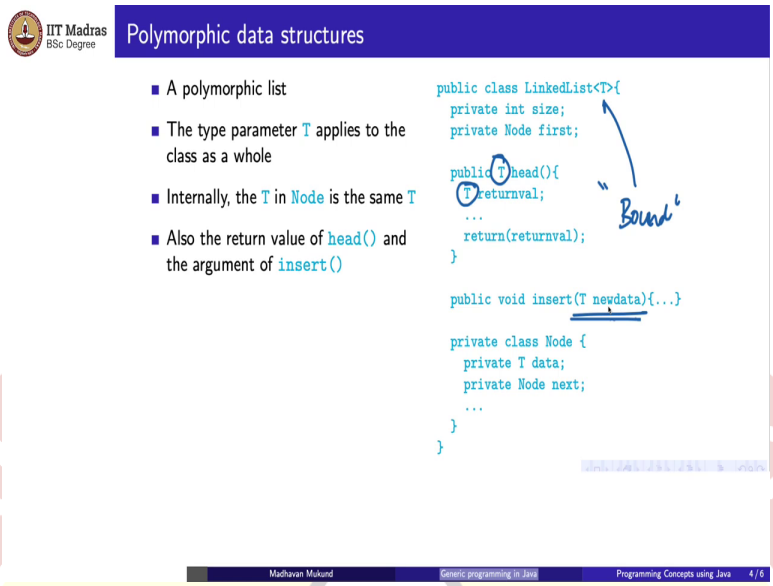
- A polymorphic list
- The type parameter T applies to the class as a whole
- Internally, the T in Node is the same T

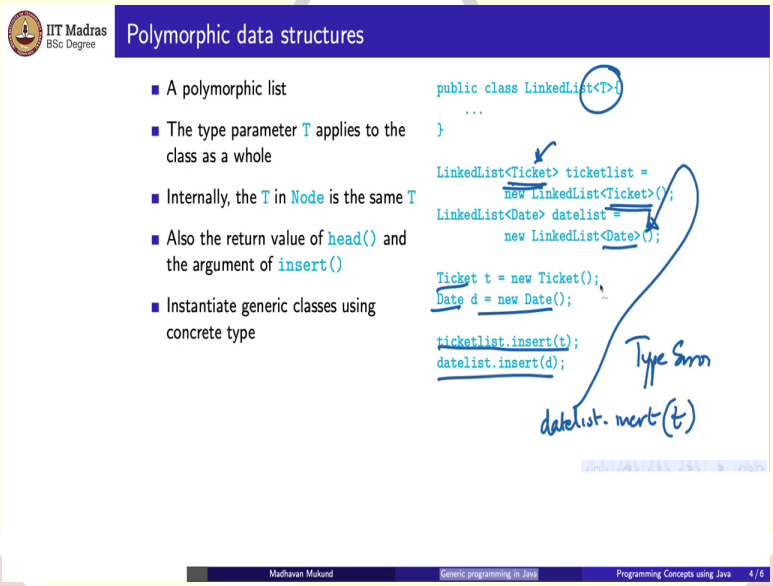
```
public class LinkedList<T>{
  private int size;
  private Node first;

  public T head(){
    T returnval;
    ...
    return(returnval);
  }

  public void insert(T newdata){...}

  private class Node {
    private T data;
    private Node next;
    ...
  }
}
```





So, now, moving forward, let us look at the other kind of requirements that we had, which was to define these polymorphic data structures. So, we wanted to define a linked list, for example, which could take arbitrary values. And earlier what we had said was it will consist of private nodes and each private node which store a data value of type object. And the problem was two things; one is that the objects need not all be the same; and the second thing was whenever we extracted a value from this list, we had to cast it back to the original type.

So, here now, we use this type quantify in a slightly different way. So, we stick it at the end of the class definition. So, here we are not associating the quantifier with the function, but with a

class. So, when we associate a quantifier with a class, we say that this is a linked list. So, you should read it as a linked list of type T. So, the earlier thing said for all T, reverse T works this way. This one is saying, this is a class which creates a linked list of type T.

Now, once I have defined this, then this class, this parameter can, like we saw for the reverse case, it is used inside in a consistent way. So, everything inside when it refers to T is referring to the same T. So, if I created a linked list of string, then throughout everything T will be a string. If I created a linked list of date, throughout everything will be a date.

So, in particular, for instance, I now have this private class node and it uses T as the name of the type for the data value. So, this T on its own does not make sense, but when I have a concrete list, I have a concrete T. So, we will talk about how to create a concrete list in a minute. But if I actually create a linked list of a particular type, then that type T is known and that type T is what gets used in every node.

And similarly, it is also the type T that is returned by the head function. So, inside here it has a type T which is the same. So, all these Ts are some sense bound. In the sense of mathematical logic, they are all bound to the same T. So, once I create this linked list of type T, the type T that I am using to create the list binds T everywhere.

So, similarly, here, when I insert now, I am only allowed to insert values which are the same type T as a list type. So, everywhere this type T is uniformly the same, and it is bound to the type that I used. So, how do we create a concrete linked list? Well, we actually use this same notation, but we pass a type. So, for instance, if I have these classes as before ticket and date, I can create a linked list which holds tickets by using linked list and ticket as a concrete class name. It is like, it is very much like calling a function.

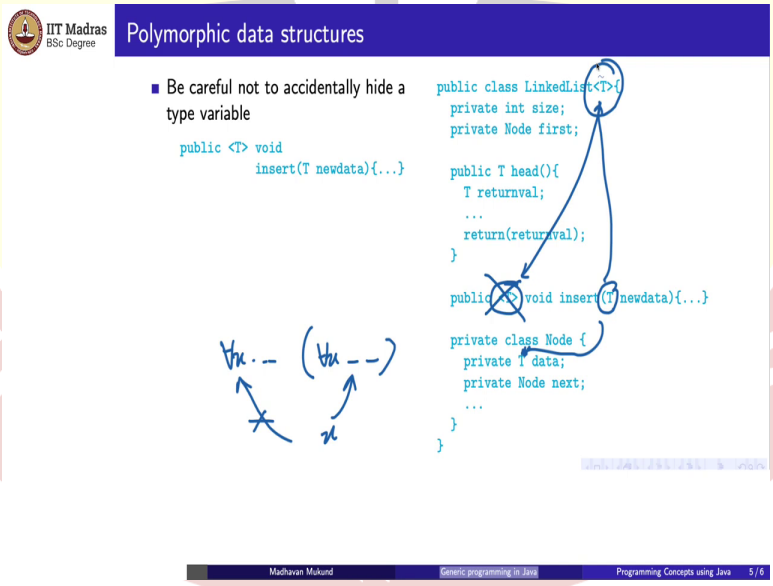
When I call a function, I pass some specific values to instantiate the parameters of the function and then that function will get called with those values. So, here, in order to create a concrete class out of this, what you might call a parameterized generic class, I pass a concrete type, and then it creates an object, a class of that type, an object of that class. So, the important thing to

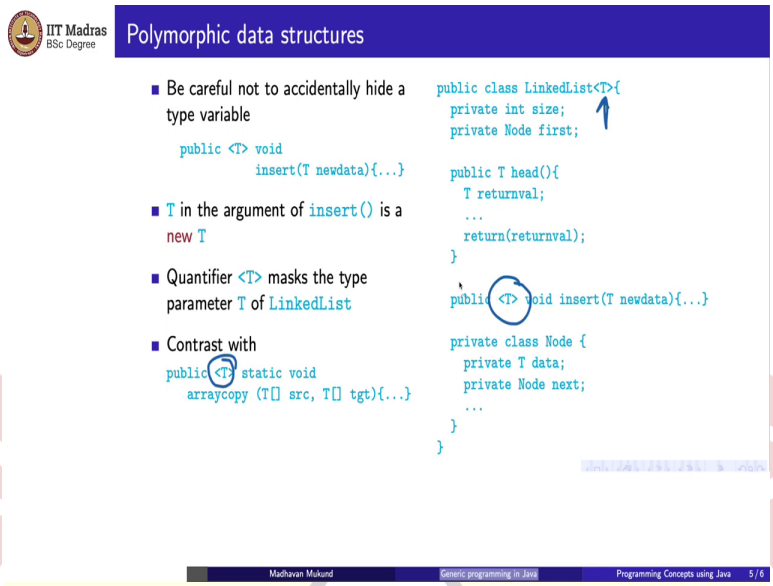
note here is that this is now a specific type. It must be a class which I have defined somewhere which is available to me in my program.

So, I say that linked list of ticket is a new linked list of this type. Linked list of date, on the other hand, is a new linked list of this type. So, in each case the T is being instantiated to a different type. The first case T is being instantiated to ticket, second case to date. And therefore, now, when I create a ticket, I can insert it into the first one, when I create a date, I can insert it into second one.

But if I try the opposite, if I try to insert a date into a ticket list, then I will be trying to insert an object of type date where an object of type ticket was required, and therefore, that will throw a type error. So, this is something that the Java compiler can now check. So, it can check that datelist.insert(t), so this is a type error, because this date list requires dates, and this t is a ticket. So, this is how we can create concrete data structures from these polymorphic types which are defined using these type variables.

(Refer Slide Time: 13:44)





Now, there is a small point that we must be careful about. Remember that when we wrote those functions earlier, we put this type quantifier as part of the function header. We said for all T, if we go back and see, for instance, when we looked at this array copy, for instance, T was a parameter of array copy. So, T was kind of used implicitly in the definition to say for every T, I can copy two arrays of type T.

So, if I do that here, then what happens is that this T, in some sense, hides that T. So, if you are at all familiar with this notation in mathematical logic, if you say for all x something, something, something and then inside this, you say for all x something, something, the x inside here is not the same as the x out there. So, there are two different x's. So, I created a kind of local x which hides the outer x and that is exactly what is happening here. So, this T is now no longer that T. So, this T in some sense now becomes a universally quantified T. I can insert any T into this thing.

And now what will happen is that when I do this, insert will accept the T but then the insert will presumably go and try to put that T into a node. And because now node is expecting a specific T, the T that insert things it is using is different from the T that node things is using and I will get a runtime error. So, you should be careful when you are writing these functions inside a parameterized class, inside a generic class, which has a type variable associated with the class, then you do not re-quantify the same variable.

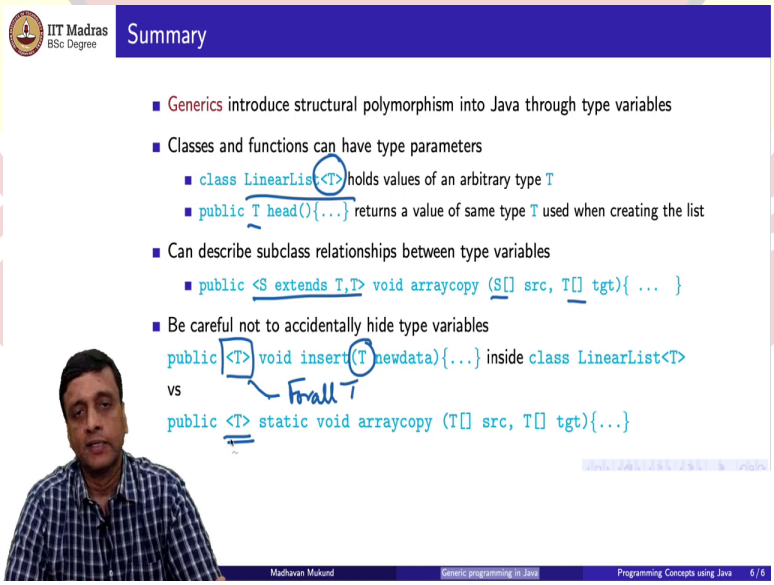
So, this T should not be quantified, because you are using this T and is implicitly quantified by the T that the class is created with. So, this T is the same T everywhere. And if you reintroduce this angle bracket T in front of a function, you are accidentally messing up things. So, this is a new T, and it masks the type parameter of linked list.

And the reason why I should be careful about it is because if you are writing static functions, for example, which do not depend on the values, so remember that this will get created, if I actually create an object of that type. If I have a static function, I do not need to create an object. If I do not create an object, I do not have a value for T.

So, when I create static functions like my array copy, I need to provide a type definition. And now it is, of course, you can be careful not to use the same name and so on whenever you need it. But, it is conventional to use letters like capital T and capital S to denote types. So, you get used to the idea that you write a function with this type quantifier.

But if you do it inside a method which belongs to the class which is associated with the instance of the class, then it is an error. So, you have to be a little careful not to accidentally introduce this type quantifier and hide the type when you are writing an instance method.

(Refer Slide Time: 16:45)



So, to summarize, what we have seen is that Java has a way of bringing structural polymorphism into the language. So, structural polymorphism is where we have some functionality which depends on some capabilities of the type. So, you must have a compatible function across different types which have the same capability without writing them specifically. So, instead of doing it through this object hierarchy, which has all sorts of problems in terms of internal consistencies and casting and so on, Java allows us to do this generics, as it is called, through type variables.

So, we can associate these type parameters with classes. And this means that this linear list can hold arbitrary values of type T. And once the type T is fixed then it also fixes the return value and the argument types of internal functions inside that. So, I can use that T in a consistent way throughout the class definition. So, the T that is used inside is implicitly given some concrete value when the class is created.

Then we also saw that you can have these functions which have two different array, two different type variables. And we can express a relationship between them by using this extends words. So, you can actually connect two different, potentially two different types by some constraint, in this case, at one extends the other.

And the last thing is that inside a class, we should always make sure that the type variable that refers to the objects of that class is inferred from the type variable that is created along with the class. So, we should not accidentally quantify that type again, because this will create a new copy of T. So, this T now becomes a new copy if we introduced this angle bracket T as part of the function definitions, and that will not be typically the intent that we had.

So, we wanted this T, the T inside insert to refer to the T with which the list was created, but instead we have accidentally made it a brand new T which can take arbitrary values, because remember, this is kind of universally contoured. This is for all T. So, we have completely changed the meaning. And the reason that this is syntactically potentially confusing is because we do need it when we do this kind of static functions here. So, one has to be mindful of that.

So, we have an overview now of how, I mean, so at this level using generics is quite straightforward. And in fact, we will see in later weeks that some of the built-in types in Java

implicitly use generics. But what we will see as we go along is that there are some limitations, because generics was kind of scrapped on as a kind of extra feature on top of the existing Java. So, there are some limitations in how genetics work because of needing to be compatible with the rest of Java. So, we will look at some of these constraints in the remaining lectures this week.